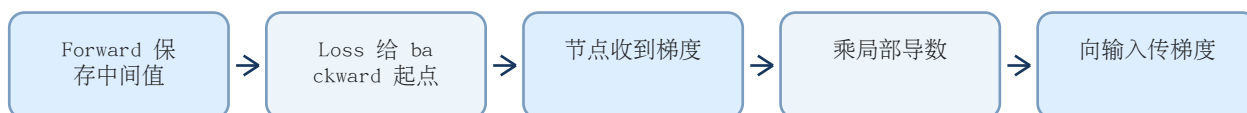


## 第23讲：Computational Graph 与局部梯度

把 Backpropagation 拆成可执行的节点规则

核心概念：【必须真正掌握】

本讲把 Chain Rule 变成真正可执行的 backward 规则。目标是：看到一个计算图节点时，知道它在 Forward 保存什么，在 Backward 收到什么梯度、乘哪个局部导数、把梯度传给哪些输入。



建议学习方式：先掌握四类标量节点规则 -> 再手工走完整图 -> 最后用数值差分检查每个参数梯度。

## 一、本讲学习目标

理解 Computational Graph 不只是“画网络结构”，而是记录每个中间量怎样由前面的量计算得到。

理解 backward 的统一模板：收到后方传来的梯度  $\times$  当前节点的局部导数 = 传给输入的梯度。

掌握 Add、Multiply、Square、ReLU 四类最小节点的 Forward / Backward 规则。

能够对  $x, w \rightarrow \text{multiply} \rightarrow \text{add bias} \rightarrow \text{ReLU} \rightarrow \text{subtract target} \rightarrow \text{square loss}$  完整手工反向传播。

理解为什么变量被多次使用时，梯度必须累加而不是覆盖。

理解参数、输入、target 都可以数学上有梯度，但优化器只更新被定义为可训练参数的变量。

把这些规则与未来 PyTorch Autograd、Transformer 计算图建立直接联系。

核心概念：第22讲到第23讲的变化

第22讲回答“为什么沿路径要用 Chain Rule”；第23讲进一步回答“如果我真的要写一个 backward 引擎，每个计算节点究竟要实现什么规则”。

## 二、【原理】Backward 节点到底在做什么？

假设一个节点做：

$$z = f(a, b)$$

Forward 时，节点读取  $a$ 、 $b$ ，算出  $z$ ；Backward 时，它会从后方收到：

$$g_z = dL/dz$$

然后分别计算：

$$dL/da = (dL/dz)(dz/da)$$

$$dL/db = (dL/dz)(dz/db)$$

核心概念：统一模板

收到的梯度  $\times$  本节点局部导数 = 传给该输入的梯度。

这里“收到的梯度”代表 Loss 对当前节点输出  $z$  的敏感度；“局部导数”只需要知道本节点自己这一步怎么算，不需要知道整个网络。

### 三、【四类最小节点】Add / Multiply / Square / ReLU

| 节点       | Forward        | 局部导数               | Backward 规则                          |
|----------|----------------|--------------------|--------------------------------------|
| Add      | $z=a+b$        | $dz/da=1, dz/db=1$ | $g_a=g_z, g_b=g_z$                   |
| Multiply | $z=a \times b$ | $dz/da=b, dz/db=a$ | $g_a=g_z \times b, g_b=g_z \times a$ |
| Square   | $z=a$          | $dz/da=2a$         | $g_a=g_z \times 2a$                  |
| ReLU     | $z=\max(0,a)$  | $1(a>0), 0(a<0)$   | $g_a=g_z \times gate$                |

科研连接：为什么只学这些小节点很有用

复杂网络最终会被分解为许多基础运算。自动微分框架并不是保存一个“整网导数公式”，而是为每种运算定义局部 backward 规则，再按计算图把它们组合起来。

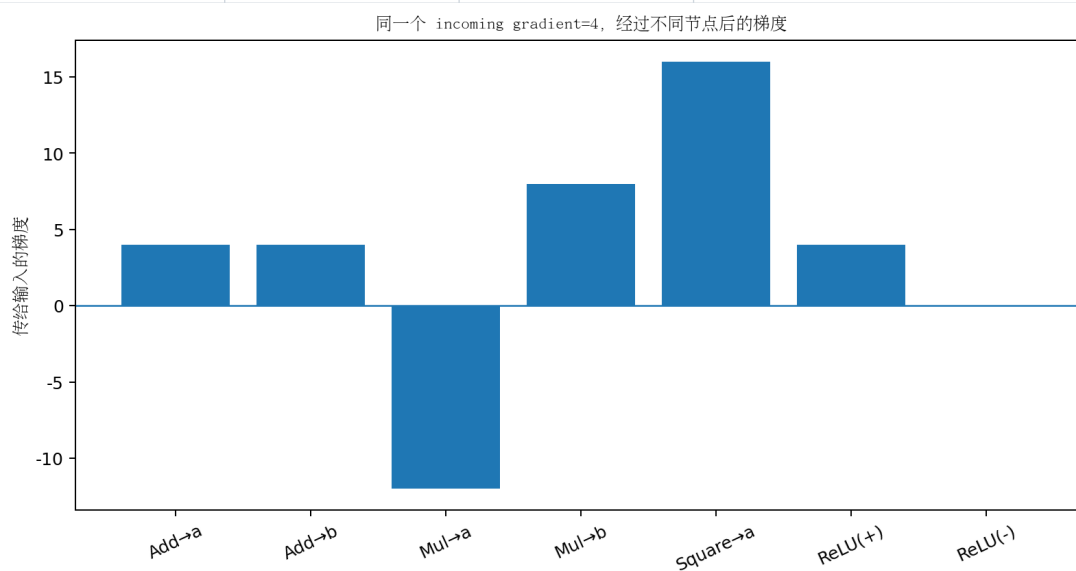
## 四、【具体数字例子】同一个 incoming gradient 经过不同节点

设后方传来同一个梯度：

$$g_z = dL/dz = 4$$

并取  $a=2$ ， $b=-3$ 。

| 节点                  | 传给a | 传给b | 为什么                |
|---------------------|-----|-----|--------------------|
| $a+b$               | 4   | 4   | 加法对两个输入的局部导数都是1    |
| $a \times b$        | -12 | 8   | 分别乘另一个输入值          |
| $a$                 | 16  | -   | $4 \times 2a = 16$ |
| $\text{ReLU}(a=2)$  | 4   | -   | 正区间局部导数=1          |
| $\text{ReLU}(b=-3)$ | 0   | -   | 负区间局部导数=0          |



数学直觉：重要直觉

Backward 不是“梯度原封不动往前传”。每经过一个节点，都会被这个节点的局部导数重新缩放、变号、放大、缩小，甚至归零。

## 五、【完整计算图】先做 Forward

我们构造一个最小但完整的训练图：

$$m = w \times x$$

$$z = m + b$$

$$a = \text{ReLU}(z)$$

$$d = a - \text{target}$$

$$L = d^2$$

取：

$$x=2, w=3, b=-1, \text{target}=4$$

| 节点 | 计算               | 结果 |
|----|------------------|----|
| m  | $3 \times 2$     | 6  |
| z  | $6 + (-1)$       | 5  |
| a  | $\text{ReLU}(5)$ | 5  |
| d  | $5 - 4$          | 1  |
| L  | $1$              | 1  |

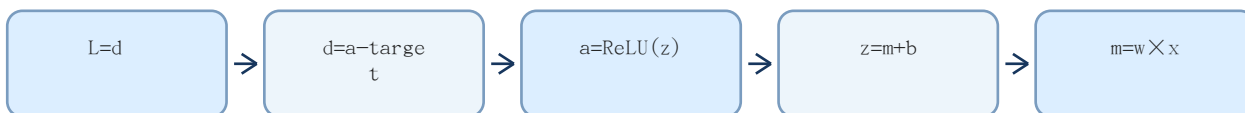
Forward 的任务就是从输入和参数出发，把所有中间值算出来并保留必要的信息。Backward 之后会用到这些中间值，例如 Multiply 节点需要知道另一个输入是多少，ReLU 需要知道输入是否大于0。

## 六、【Backward 起点】为什么先写 $dL/dL = 1$ ?

Loss 对它自己的变化率当然是:

$$dL/dL = 1$$

这就是 reverse-mode autodiff 的“种子梯度”。从这里开始往前:



每走过一个节点，就应用一次本节点的 backward 规则。

数学直觉：别把 `seed=1` 神秘化

如果  $L$  自己增加 0.01，那么  $L$  增加 0.01；因此“ $L$  对  $L$  的导数”就是 1。它只是给反向传播一个起点。

## 七、【完整手算】从 L 一步一步传回 w、b、x

第1步: Square 节点  $L=d$  。

$$g_d = 1 \times 2d = 2$$

第2步: Subtract 节点  $d=a-\text{target}$ 。

$$g_a = 2 \times 1 = 2$$

$$g_{\text{target}} = 2 \times (-1) = -2$$

第3步: ReLU 节点。因为  $z=5>0$ :

$$g_z = 2 \times 1 = 2$$

第4步: Add 节点  $z=m+b$ 。

$$g_m = 2, g_b = 2$$

第5步: Multiply 节点  $m=w \times x$ 。

$$g_w = g_m \times x = 2 \times 2 = 4$$

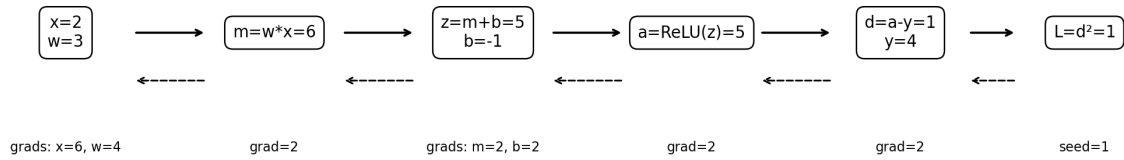
$$g_x = g_m \times w = 2 \times 3 = 6$$

核心概念: 最终答案

$dL/dw=4$ ,  $dL/db=2$ ,  $dL/dx=6$ 。整个 backward 没有使用一个“神秘的大公式”，只是每个小节点执行自己的局部规则。

## 八、【图解】同一张图里的 Forward 与 Backward

Forward: 实线从左到右算数值; Backward: 虚线从右到左传梯度



实线方向代表 Forward 的数值计算; 虚线代表 Backward 的梯度信息反向传播。

| Forward 存什么           | Backward 为什么需要 |
|-----------------------|----------------|
| Multiply 的两个输入 $w, x$ | 局部导数分别需要另一个输入  |
| ReLU 输入 $z$           | 要判断 gate 是1还是0 |
| Square 输入 $d$         | 局部导数是 $2d$     |
| 图的依赖关系                | 决定梯度应该传给哪些父节点  |

科研连接: 这就是 autograd tape 的直觉

现代框架会在 Forward 构建/记录计算依赖和必要上下文, Backward 再沿图的反方向调用各运算的梯度规则。不同框架具体实现不同, 但最低层直觉就是这样。

## 九、【为什么 target 也有数学梯度，但通常不更新？】

在  $d=a-target$  中，我们算出了：

$$dL/dtarget = -2$$

这并不意味着训练要“修改正确答案”。数学上的导数与“是否是可训练参数”是两回事。

| 对象             | 可能有梯度吗 | 通常是否被 optimizer 更新 |
|----------------|--------|--------------------|
| Weight / Bias  | 有      | 是                  |
| 输入 x           | 可以有    | 普通监督训练中通常不是        |
| Target / Label | 数学上可求  | 否                  |
| 中间激活值          | 有      | 不是独立可训练参数          |

科研连接：科研连接

以后你会遇到 adversarial examples、input optimization、prompt embedding optimization 等场景，此时输入本身也可能被当作优化变量。是否更新某个量，取决于实验设定，不取决于“它能不能求导”。

## 十、【梯度累加】为什么变量被使用两次时不能覆盖？

$$L = x^2 + 3x$$

$x$  有两条路径到达 Loss:



在  $x=2$ : 第一条路径贡献4, 第二条贡献3, 所以:

$$dL/dx = 4 + 3 = 7$$

实现 backward 时, 同一个变量可能从多个后继节点收到梯度。因此应该:

$$\text{grad}[x] += \text{contribution}$$

而不是:

$$\text{grad}[x] = \text{contribution}$$

常见误区 / 风险: 这是自动微分实现中的关键细节

如果共享变量、Residual、分支网络里把梯度“覆盖”而不是“累加”, 结果就会错误。PyTorch 的参数 `.grad` 在多次 backward / 多个路径上也涉及累加语义。

## 十一、【ReLU 关闭路径】一个局部导数怎样截断梯度？

把  $w$  改成  $-3$ ，其他保持不变：

$$z = (-3) \times 2 - 1 = -7$$

$$a = \text{ReLU}(-7) = 0$$

此时 ReLU 局部导数为0。即使后方 Loss 很大：

$$L = (0-4)^2 = 16$$

反传经过 ReLU 后：

$$g_z = g_a \times 0 = 0$$

于是继续往前：

$$g_w=0, g_b=0, g_x=0$$

常见误区 / 风险：非常重要

“Loss 很大” 不保证 “所有参数都有大梯度”。梯度还要看路径中每个节点的局部导数。后面理解 dead ReLU、vanishing gradient 时会反复用到这点。

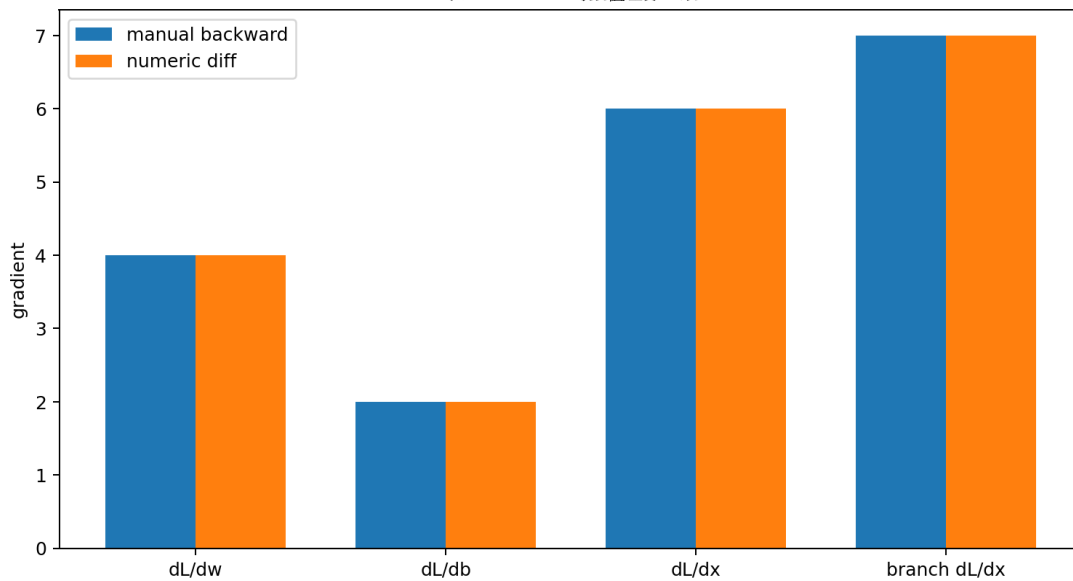
## 十二、【数值差分检查】手工 backward 对不对？

对  $w$ 、 $b$ 、 $x$  分别做 central difference:

$$dL/dq \approx [L(q+\epsilon) - L(q-\epsilon)] / (2\epsilon)$$

| 梯度         | 手工 backward | 数值差分     | 绝对误差        |
|------------|-------------|----------|-------------|
| $dL/dw$    | 4.000000    | 4.000000 | 约 $2.6e-11$ |
| $dL/db$    | 2.000000    | 2.000000 | 约 $7.6e-11$ |
| $dL/dx$    | 6.000000    | 6.000000 | 约 $1.3e-10$ |
| 分支 $dL/dx$ | 7.000000    | 7.000000 | 极小          |

手工 backward 与数值差分一致



科研连接: Gradient Check 的意义

它不是为了替代 Backprop 训练，而是用一个慢但直观的近似方法，检查你自己写的局部 backward 或新算子实现有没有错。

### 十三、【一次最小参数更新】Backward 输出最终给谁？

Backward 算出：

$$dL/dw=4, dL/db=2$$

如果学习率：

$$lr = 0.05$$

最简单 Gradient Descent 更新：

$$w\_new = 3 - 0.05 \times 4 = 2.8$$

$$b\_new = -1 - 0.05 \times 2 = -1.1$$

重新 Forward 后：

$$\text{Loss: } 1.00 \rightarrow 0.25$$

核心概念：注意课程顺序

本讲只是让你看到“backward 产出的梯度最终会被参数更新使用”。真正的 Gradient Descent、SGD、Momentum、Adam/AdamW 会在后面的优化器阶段系统学习。

十四、【与 PyTorch Autograd 的关系】先理解，再交给框架

| 你手工做的事                | 以后 PyTorch 对应概念                    |
|-----------------------|------------------------------------|
| 保存 Tensor 与运算依赖       | 动态计算图 / autograd graph             |
| Square / ReLU 等节点局部规则 | 对应算子的 backward 定义                  |
| 从 $dL/dL=1$ 开始        | <code>loss.backward()</code> 的反向起点 |
| 共享变量的贡献相加             | gradient accumulation              |
| 得到 $w, b$ 的梯度         | <code>parameter.grad</code>        |
| 用梯度改参数                | <code>optimizer.step()</code>      |

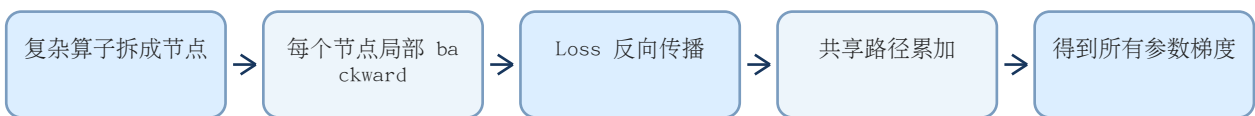
科研连接：为什么第27讲才正式进入 PyTorch

现在先把 backward 的每一步手工跑通，之后看到 `loss.backward()` 时，你知道它自动替你做了哪些节点级运算，而不是把 Autograd 当魔法。

## 十五、【与 Transformer / LLM 的关系】

| 组件                   | Backward 时发生什么                |
|----------------------|-------------------------------|
| $Q=XW_Q$             | 矩阵乘法节点把梯度分别传回 $X$ 和 $W_Q$     |
| $QK^T$               | 乘法/矩阵乘法节点把梯度传回 $Q$ 、 $K$      |
| Softmax              | 用 Softmax 的局部 Jacobian 规则传播梯度 |
| Residual: $y=F(x)+x$ | Add 节点把梯度复制到两个分支，之后在 $x$ 处累加  |
| LayerNorm / RMSNorm  | 按其内部算子链继续传播                   |
| Cross Entropy        | 从最终 logits 端产生反向信号            |

真实 LLM 的图巨大，但核心不变：



科研连接：最低必要科研理解

以后读论文里的 `stop-gradient`、`detach`、`gradient checkpointing`、`custom autograd`、`gradient clipping` 等概念时，都要先问：它改变了计算图中的哪条梯度路径？

## 十六、【常见误区】

常见误区 / 风险：误区1：计算图就是模型结构图

模型结构图更关注模块；计算图关注一次具体计算中每个中间量的依赖关系和运算节点。

常见误区 / 风险：误区2：Backward 每个节点只需要知道最终 Loss 公式

恰恰相反，每个节点只需知道收到的梯度和本地局部导数，不需要了解整个网络。

常见误区 / 风险：误区3：梯度经过 Add 会被“分成一半”

不会。 $z=a+b$  对  $a, b$  的局部导数都是1，因此同一个 incoming gradient 会完整传给两个输入。

常见误区 / 风险：误区4：一个变量收到多个梯度时任选一个即可

错误。所有路径的一阶贡献必须累加。

常见误区 / 风险：误区5：有梯度的量都会被 optimizer 更新

错误。可求导和可训练是两件事。

常见误区 / 风险：误区6：Loss 大就应该所有参数梯度都大

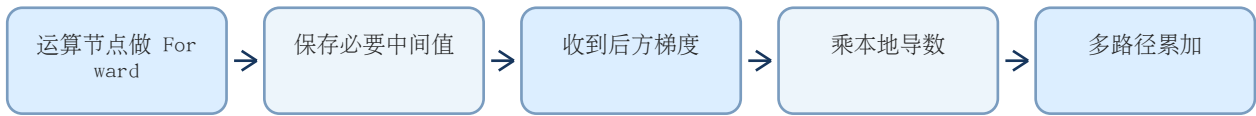
错误。局部导数可能很小或为0，某些路径可能没有梯度。

十七、【代码实验】本讲做什么？

| 实验                | 要验证的问题                     | 结果                                  |
|-------------------|----------------------------|-------------------------------------|
| 1. 节点规则           | 不同节点怎样变换 incoming gradient | Add复制；Mul乘另一个输入；Square乘2a；ReLU做gate |
| 2. 完整图            | 能否手工完成一次 backward          | 得到 dw=4, db=2, dx=6                 |
| 3. Gradient Check | 手算是否正确                     | 与数值差分一致                             |
| 4. ReLU关闭         | Loss大但梯度能否为0               | 该路径梯度为0                             |
| 5. 共享变量           | 多路径是否需要累加                  | 4+3=7                               |
| 6. 参数更新           | 梯度怎样被 optimizer 使用         | 一步后 Loss 1->0.25                    |

```
# 一个最小 multiply backward
m = w * x
# 后方传来 g_m = dL/dm
g_w = g_m * x
g_x = g_m * w
```

## 十八、本讲知识闭环



到这里，你已经把 Backpropagation 从一句定义拆成了真正能执行的节点规则。

核心概念：下一讲

第24讲：一个神经元的 Backpropagation——把 Weight、Bias、Input 的梯度从头到尾手算一遍，并进一步理解矩阵形式为什么只是同一规则的批量版本。